AdvR 11 – R6, S4 & OO Trade-offs

R6

What Is R6?

R6 is **encapsulated OOP** with **reference semantics**:

- Methods belong to **objects**, called with `object$method()`
- Objects are **mutable** — modified in place, not copied

```
Accumulator <- R6Class("Accumulator", list(
  sum = 0,
  add = function(x = 1) {
    self$sum <- self$sum + x
    invisible(self)
  }
))
x <- Accumulator$new()
x$add(4)
x$sum
```

```
#> [1] 4
```

One function: `R6::R6Class()`. That's all you need.

$initialize() and $print()

`$initialize()` validates inputs on construction. `$print()` customises display:

```
Person <- R6Class("Person", list(
  name = NULL, age = NA,
  initialize = function(name, age = NA) {
    stopifnot(is.character(name), length(name) == 1)
    stopifnot(is.numeric(age), length(age) == 1)
    self$name <- name; self$age <- age
  },
  print = function(...) {
    cat("Person:", self$name,
      "(age", self$age, ")\n")
    invisible(self)
  }
))
hadley <- Person$new("Hadley", age = 38)
hadley
```

```
#> Person: Hadley (age 38 )
```

Method Chaining

Side-effect methods return `invisible(self)` → enables chaining:

```
x <- Accumulator$new()
x$add(10)$add(20)$add(30)$sum
```

```
#> [1] 60
```

Equivalent to piping in functional style. Readable with line breaks:

```
x$
  add(10)$
  add(20)$
  sum
```

Inheritance

Use `inherit` to derive from an existing class. Override methods, access parent with `super$`:

```
AccumulatorChatty <- R6Class("AccumulatorChatty",
  inherit = Accumulator,
  public = list(
    add = function(x = 1) {
      cat("Adding", x, "\n")
      super$add(x = x)
    }
  )
)
x2 <- AccumulatorChatty$new()
x2$add(10)$add(1)$sum
```

```
#> Adding 10
#> Adding 1
```

```
#> [1] 11
```

`super$` is analogous to `NextMethod()` in S3.

Private Fields and Methods

`private` hides internals — accessible only via `private$`:

```
Person <- R6Class("Person",
  public = list(
    initialize = function(name, age = NA) {
      private$name <- name; private$age <- age
    },
    print = function(...) {
      cat("Person:", private$name,
        "(age", private$age, ")\n")
      invisible(self)
    }
  ),
  private = list(name = NULL, age = NA)
)
p <- Person$new("Hadley", 38)
p
```

```
#> Person: Hadley (age 38 )
```

```
p$name   # NULL -- can't access private field
```

```
#> NULL
```

Active Fields

Active fields look like fields but are computed via functions:

```
Rando <- R6Class("Rando", active = list(
  random = function(value) {
    if (missing(value)) runif(1)
    else stop("Can't set `$random`", call. = FALSE)
  }
))
x <- Rando$new()
c(x$random, x$random, x$random)
```

```
#> [1] 0.15121799 0.50774362 0.09427004
```

Useful for **read-only** fields or fields with validation on write.

Reference Semantics

R6 objects are **not** copied — all references point to the same object:

```
x <- Accumulator$new()
y <- x       # y and x are the SAME object
y$add(10)
x$sum         # x is also modified!
```

```
#> [1] 10
```

Use `$clone()` for an independent copy:

```
x <- Accumulator$new()
y <- x$clone()
y$add(10)
x$sum         # x is unaffected
```

```
#> [1] 0
```

For objects containing other R6 objects, use `$clone(deep = TRUE)`.

Exercises (R6)

1. Create a bank account R6 class with `$deposit()` and `$withdraw()`. Create a subclass that throws an error on overdraft.
2. Why can't you model a bank account with an S3 class?
3. What base type are R6 objects built on? What attributes do they have?

Solutions (R6)

1.

```
BankAccount <- R6Class("BankAccount", list(
  balance = 0,
  deposit  = function(x) {
    self$balance <- self$balance + x; invisible(self)
  },
  withdraw = function(x) {
    self$balance <- self$balance - x; invisible(self)
  }
))
SafeAccount <- R6Class("SafeAccount",
  inherit = BankAccount,
  public = list(
    withdraw = function(x) {
      if (self$balance - x < 0) stop("Overdraft!")
      super$withdraw(x)
    }
  )
)
sa <- SafeAccount$new(); sa$deposit(100)$withdraw(150)
```

2. S3 uses copy-on-modify: `$withdraw()` must return a new object and the caller must reassign (`s <- withdraw(s, 50)`). This is awkward ("threading state"). R6 modifies in place, so `s$withdraw(50)` just works.
3. R6 objects are **environments** (base type `environment`) with a `class` attribute containing the R6 class hierarchy:

```
x <- Person$new("Test", 30)
typeof(x)
```

```
#> [1] "environment"
```

```
class(x)
```

```
#> [1] "Person" "R6"
```

S4

S4 Overview

S4 is **formal functional OOP** — like S3 but stricter:

```
methods::setClass("S4Person",
  slots = c(name = "character", age = "numeric")
)
john <- methods::new("S4Person",
  name = "John", age = 30)
john@name   # @ accesses slots (like $ for S3)
```

```
#> [1] "John"
```

Three key functions: `setClass()`, `setGeneric()`, `setMethod()`.

S4 Generics and Methods

```
setGeneric("greet", function(x) {
  standardGeneric("greet")
})
setMethod("greet", "S4Person", function(x) {
  paste0("Hello, ", x@name, "!")
})
greet(john)
```

```
#> [1] "Hello, John!"
```

S4 supports **multiple inheritance** and **multiple dispatch** (dispatch on multiple arguments).

S4 Key Differences from S3

Feature	S3	S4
Define class	`structure()` + attribute	`setClass()` with slots
Access fields	`$`	`@`
Create generic	`UseMethod()`	`setGeneric()` + `standardGeneric()`
Define method	`generic.class()`	`setMethod()`
Inheritance	Class vector	`contains` argument
Multiple dispatch	No	Yes
Validation	Manual	`setValidity()`

S4 is used extensively by **Bioconductor**.

OO Trade-offs

S4 vs. S3

S3: simple, flexible, good for individuals and small teams.
S4: formal, strict, good for large teams and complex systems.

	S3	S4
Learning curve	Low	High
Upfront design	Minimal	Required
Guarantees	Conventions	Enforced
Best for	Most R code	Bioconductor, Matrix

Default to S3 unless you have a compelling reason for S4.

R6 vs. S3: Namespacing

S3 generics live in the **global** namespace → naming conflicts:

```
plot(data)       # plot some data
plot(bank_heist) # plot a crime?
```

R6 methods live in the **object** → no conflict:

```
data$plot()
bank_heist$plot()
```

R6 methods can also have more specific argument names (no need for `x`, `...`).

R6 vs. S3: Threading State

S3 problem: returning a value AND modifying an object:

```
# S3 stack: awkward
out <- pop(s)     # returns list(item, stack)
s <- out$x        # must reassign!
```

R6 solution: modify in place, return only the value:

```
# R6 stack: natural
value <- s$pop() # modifies s AND returns value
```

Reference semantics solve "threading state" — but make code harder to reason about.

When to Use Which System

Scenario	Recommended
Extending base R / tidyverse	**S3**
Most R packages	**S3**
Mutable state (connections, caches)	**R6**
Web API wrappers, databases	**R6**
Bioconductor, complex hierarchies	**S4**

General rule: start with S3. Use R6 when you need mutability. Use S4 when formality pays off.

Exercises (Trade-offs)

1. How does method chaining in R6 relate to the pipe (|>) in S3?
2. Name two advantages of S3's global namespace for generics.

Solutions (Trade-offs)

1. Both allow sequential operations. **Method chaining** (`x$add(1)$add(2)$sum`) reads left-to-right like a pipe. **Pipe** (`x |> add(1) |> add(2)`) does the same for functional code. The key difference: chaining requires `invisible(self)` returns; piping works with any function that returns a new object.
2. a. **Uniform API:** `summary()`, `print()`, `predict()` work on any object — users learn one verb for many types. (b) **Discoverability:** tab-completion on a generic shows all available methods; with R6, you must know the object's class first.